



Compte rendu TP4 FPGA: Contrôle d'un moteur pas à pas avec un capteur ultrason

Abdelouahab BELARBI et Katia BELAIDI

13 juin 2026



Table des matières

1	Introduction	4
2	Objectifs	4
3	Moteur pas à pas	5
3.1	Principe	6
3.2	circuit	6
3.3	code VHDL	7
3.4	Timing simulation	11
4	Capteur ultrason	13
4.1	Principe	13
4.2	circuit	14
4.3	code VHDL	14
4.4	Timing simulation	19
5	Communication entre le capteur et l'actionneur	20
5.1	Principe	20
5.2	circuit	20
5.3	code VHDL	21
5.4	Timing simulation	24
5.5	RTL viewer, Chip planner	24
6	Implémentation sur la carte DE10 Lite	27
7	Conclusion	28
8	Sources :	30

Note : Gemini a été utilisé pour corriger la ponctuation et les fautes de grammaire dans les textes d'explication. Le prompt utilisée étaient le suivante :

- *Fix any spelling issues, punctuation and explain the mistakes underneath. Make minimal changes to the vocabulary, only if necessary.*

1 Introduction

C'est le travail final qui nous a été assigné dans le cours de FPGA de cette année. Contrairement à la plupart des travaux que nous avons réalisés lors des derniers TP, celui-ci repose fortement sur l'autonomie et la recherche.

On nous a fourni un moteur pas à pas, avec un driver de contrôle pour celui-ci, un capteur à ultrasons, et plus important encore une nouvelle carte FPGA, la DE10-Lite. Nous avons été chargés de rechercher des informations sur tous ces composants, de comprendre comment contrôler chacun d'eux indépendamment, puis d'établir une connexion entre eux avec un scénario de notre choix.

Nous avons d'abord effectué toutes ces manipulations sur la fidèle DE0 car nous nous étions familiarisés avec elle ; puis, un assez bon exercice a été de trouver un moyen de traduire ces résultats sur la DE10-Lite, avec une version plus récente de Quartus.

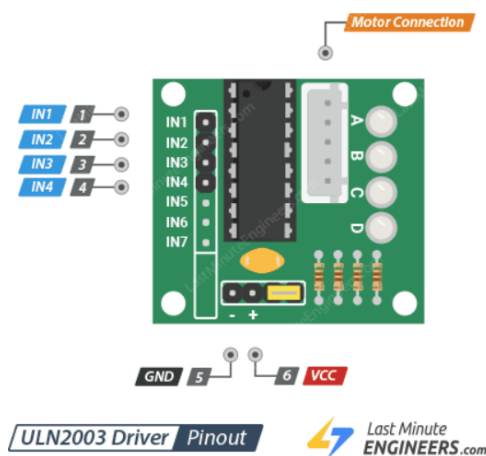
Le scénario que nous avons choisi était assez simple : le moteur représentera une porte, s'ouvrant et se fermant en fonction de la distance à laquelle se trouve un objet.

2 Objectifs

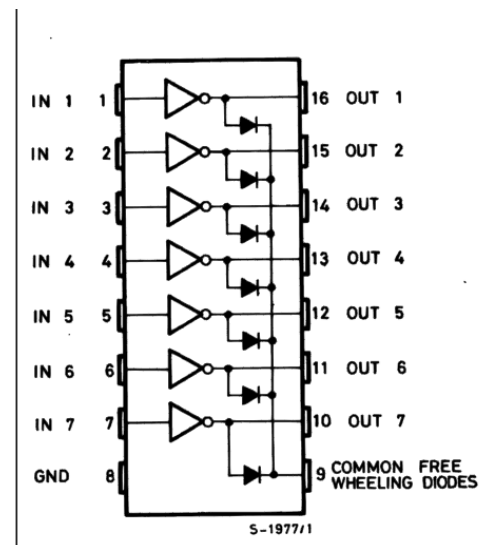
1. Pilotage du moteur pas à pas : changement de direction et variation de la vitesse
2. Pilotage du capteur : définition de conditions avec une LED, affichage de la distance sur un afficheur 7 segments.
3. Établissement de la relation entre le capteur et l'actionneur.
4. Traduction vers la carte DE10-Lite.

3 Moteur pas à pas

On nous a fourni un moteur unipolaire, le 28BYJ-48, qui fonctionne en 5V DC et possède 4 phases (nous expliquerons les phases dans la section suivante). Pour le contrôler, on nous a également fourni un driver, le ULN2003, ce qui nous permet d'envoyer des informations depuis la DE0 vers notre moteur.



(a) driver ULN2003



(b) ULN2003 chip

FIGURE 1 – Chip planner scomp vs. scompplus

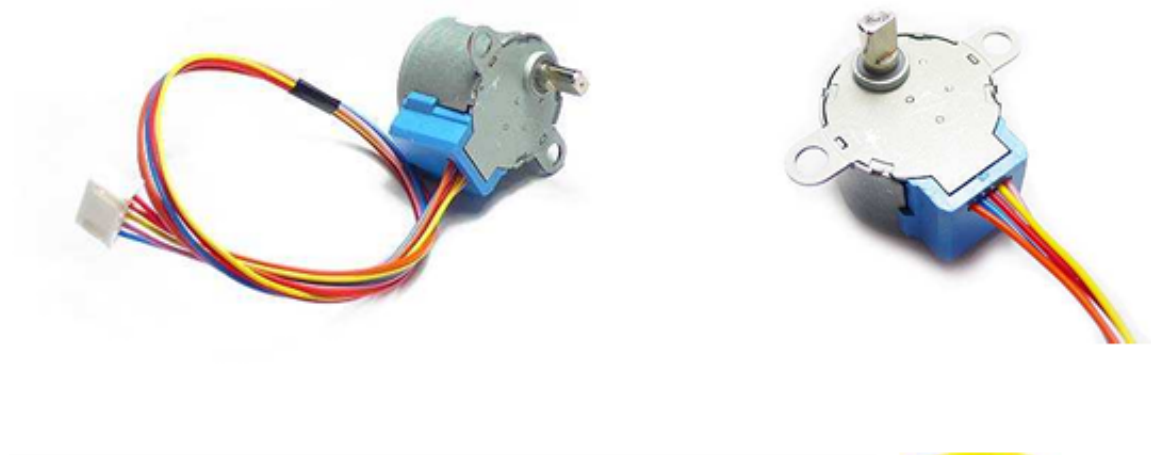


FIGURE 2 – step motor 28BYJ-48

3.1 Principe

Nous avons acquis une bonne compréhension des moteurs pas à pas grâce au cours d'électrotechnique de M. Manceau. L'idée de base est que le moteur est composé d'une partie statique (stator) et d'une partie mobile (rotor) ; le premier est constitué de 4 bobines qui représentent les 4 phases de notre moteur, et le second est représenté par un aimant. Les deux parties interagissent entre elles via des champs magnétiques, ce qui provoque le mouvement circulaire.

C'est ici que le driver intervient, en nous permettant de contrôler quand et où nous envoyons le courant à travers l'une des bobines, ce qui correspond au mode "wave drive" de notre moteur. Cela se traduit par une sorte de code binaire sur 4 bits : nous allumons une bobine à la fois, donc la première serait 1000 et la dernière 0001. Pour inverser la rotation, il suffit de commencer par le dernier code précédent et de terminer par le premier.

Nous avons une variable déterminante de plus, qui est la fréquence à laquelle ces étapes que nous avons introduites défilent à travers leurs états. Une fréquence plus élevée signifie une vitesse plus élevée.

3.2 circuit

Le moteur est relié au driver par 5 fils : 4 d'entre eux sont connectés à chaque bobine via les broches de sortie de la puce principale du driver (OUT 1 à 4), et le cinquième fil, le rouge, est une entrée commune 5V, fournissant la tension selon la bobine active.

Comme nous allons changer de sens de rotation, nous avons configuré 3 switches pour agir comme le sélecteur de vitesse en code binaire, nous donnant ainsi plusieurs modes de vitesse à faire

défiler. Il y a aussi un switch pour choisir la direction. Nous faisons bien sûr appel à une horloge (clock) également, elle sert toujours de référence.

Le driver possède une broche 5V et une broche GND, que nous connectons directement au FPGA, sur les broches GPIO; nous avons trouvé tous leurs emplacements en consultant le manuel de la DE0. La puce principale du driver a 4 entrées (IN 1 à 4), chacune connectée à une entrée (nous avons choisi les broches 2, 4, 6 et 8 car elles sont alignées). Cette connexion passe par la breadboard, uniquement parce que nous n'avons pas assez de câbles femelle-femelle.

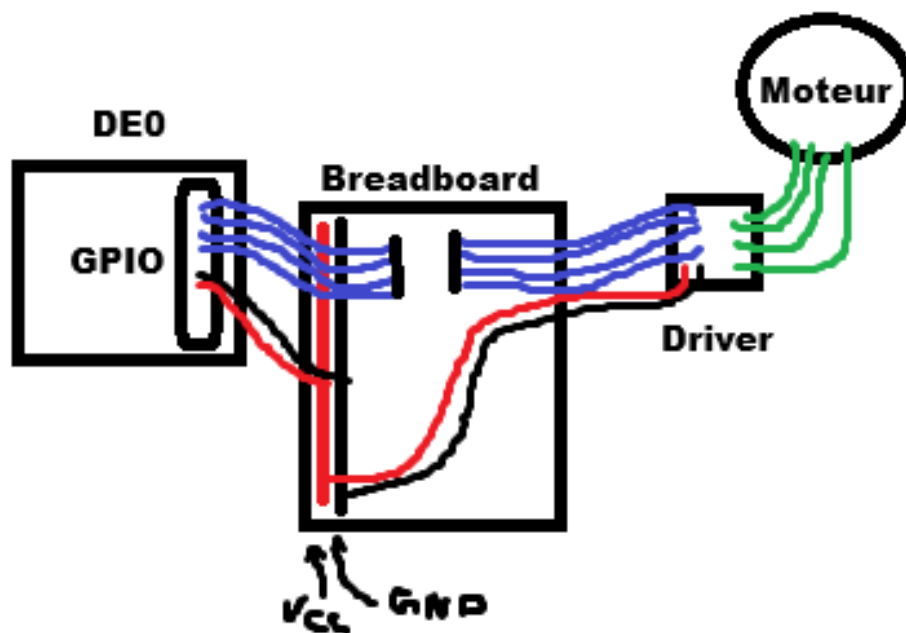


FIGURE 3 – Circuit Moteur

3.3 code VHDL

En cherchant des ressources sur le sujet des moteurs pas à pas, nous sommes tombés sur une vidéo YouTube qui présentait des

vitesses variables sur le moteur ; nous nous sommes donc inspirés de leur travail pour la suite.

Quelque chose de nouveau que nous avons appris en code VHDL est l'utilisation des *processes*, où nous séparons notre circuit en morceaux plus abordables avec des fonctions spécifiques. Nous utilisons cela au début de notre code, en nommant ce premier processus « speed-degree », où nous utilisons des instructions if-else pour définir différentes fréquences, et donc différentes vitesses. Nous nommons notre entrée sur 3 bits « speed » en veillant à l'injecter dans le processus. Pour chaque combinaison différente, nous attribuons à la variable que nous avons définie, « stop », une valeur différente allant de 5,76 M à 360 k. Cette variable représente le délai entre les pas : ainsi, pour la valeur la plus grande, le moteur se déplace le plus lentement, et la valeur la plus courte le rend plus rapide. Les valeurs choisies sont simplement des valeurs en 2^n (pour l'optimisation matérielle). stop = 0 arrête le système.

Nous nommons notre second processus « direction » et y insérons les ports « clock » et « direction-rotation », ce dernier port étant juste un switch (zéro ou un).

Nous commençons par définir un compteur « cnt » qui servira d'agent de comparaison et qui s'incrémentera avec l'horloge. Nous ouvrons une instruction if qui se répète à chaque front montant de l'horloge (rising edge), menant à une deuxième instruction if confirmant dans quelle direction le moteur va tourner. Directement après, nous définissons l'incrément de notre compteur, menant à une troisième instruction if qui comporte plusieurs cas. L'idée est de comparer « cnt » à différents quarts du délai choisi dans le processus 1 : nous commençons par le premier quart pour exécuter la première commande qui activera la bobine 1, puis le deuxième quart (la moitié du délai), jusqu'à comparer « cnt » au

	Node Name	Location
1	clock	PIN_G21
2	direction_rotation	PIN_J6
3	motor_out[3]	PIN_AB16
4	motor_out[2]	PIN_AA16
5	motor_out[1]	PIN_AB15
6	motor_out[0]	PIN_AB14
7	speed[2]	PIN_D2
8	speed[1]	PIN_E4
9	speed[0]	PIN_E3
10	<<new node>>	

FIGURE 4 – pin planner "Motor"

délai complet. C'est une règle générale en programmation d'éviter d'utiliser des délais qui bloquent l'exécution; cette méthode permet un mouvement fluide. Quand le compteur dépasse le délai, il est remis à zéro pour pouvoir recommencer. La même chose se produit, mais en commençant par la bobine 4, si le switch « direction-rotation » est basculé.

Voici une vidéo montrant le fonctionnement du programme.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3
4
5  ENTITY Moteur is
6  PORT ( clock: in std_logic;
7        direction_rotation : in std_logic;
8        speed : in std_logic_vector(2 downto 0);
9        motor_out: out std_logic_vector(3 downto 0));
10 END Moteur;
11
12
13 architecture Behavioral of Moteur is
14 signal stop : integer := 0;
15
16 begin
17 Speed_degree : process(speed)
18
19 begin
20
21

```

```

22 if (speed = "000") then -- degree 1
23     stop <= 0;
24     elsif (speed = "001") then -- degree 2
25         stop <= 5_760_000;
26     elsif (speed = "010") then -- degree 3
27         stop <= 2_880_000;
28     elsif (speed = "011") then -- degree 4
29         stop <= 1_440_000;
30     elsif (speed = "100") then -- degree 5
31         stop <= 720_000;
32     elsif (speed = "101") then -- degree 6
33         stop <= 360_000;
34 end if;
35
36 end process;
37
38 direction : process(clock, direction_rotation)
39     variable cntr : integer := 0;
40
41 begin
42
43     if rising_edge(clock) then
44         if direction_rotation = '0' then
45
46             cntr := cntr + 1;
47
48             if (cntr < stop/4) then
49                 motor_out <= "0001";
50             elsif (cntr < stop/2) then
51                 motor_out <= "0010";
52             elsif (cntr < stop*3/4) then
53                 motor_out <= "0100";
54             elsif (cntr < stop) then
55                 motor_out <= "1000";
56             else
57                 cntr := 0;
58             end if;
59
60             else
61                 cntr := cntr + 1;
62
63                 if (cntr < stop/4) then
64                     motor_out <= "1000";
65                 elsif (cntr < stop/2) then
66                     motor_out <= "0100";
67                 elsif (cntr < (3*stop)/4) then
68                     motor_out <= "0010";
69                 elsif (cntr < stop) then
70                     motor_out <= "0001";
71                 else
72                     cntr := 0;
73                 end if;
74
75             end if;
76
77         end if;

```

```

78 end if;
79 end process;
80
81 end behavioral;

```

Listing 1 – Motor VHDL code

3.4 Timing simulation

Une adaptation a été nécessaire pour simuler le comportement de notre moteur : l'ajustement de l'échelle temporelle. Comme nous l'avons observé lors de nos simulations précédentes, il est impératif de réduire la durée des délais dans le code VHDL pour les tests fonctionnels. En effet, les délais réels (plusieurs millisecondes) demanderaient des ressources de calcul trop importantes et un temps de simulation excessivement long.

Pour cette raison, nous avons temporairement réduit la valeur de la constante de délai à 50 cycles au lieu des millions de cycles utilisés pour l'application physique. Cette modification nous permet de visualiser clairement le comportement suivant dans le simulateur :

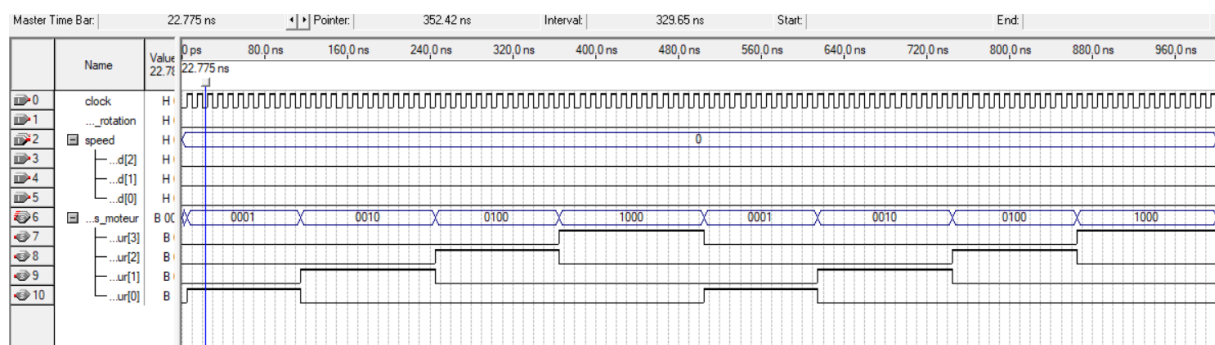


FIGURE 5 – Timing simulation "Motor"

Nous obtenons le résultat attendu : un décalage séquentiel cyclique sur les quatre ports de sortie (motor-out[3..0]). Ce motif de bits confirme le bon fonctionnement de notre automate d'états. C'est exactement le même signal que nous avons pu observer lors

des tests physiques du moteur à l'aide d'un oscilloscope, validant ainsi la transition entre le modèle théorique et la réalité matérielle.

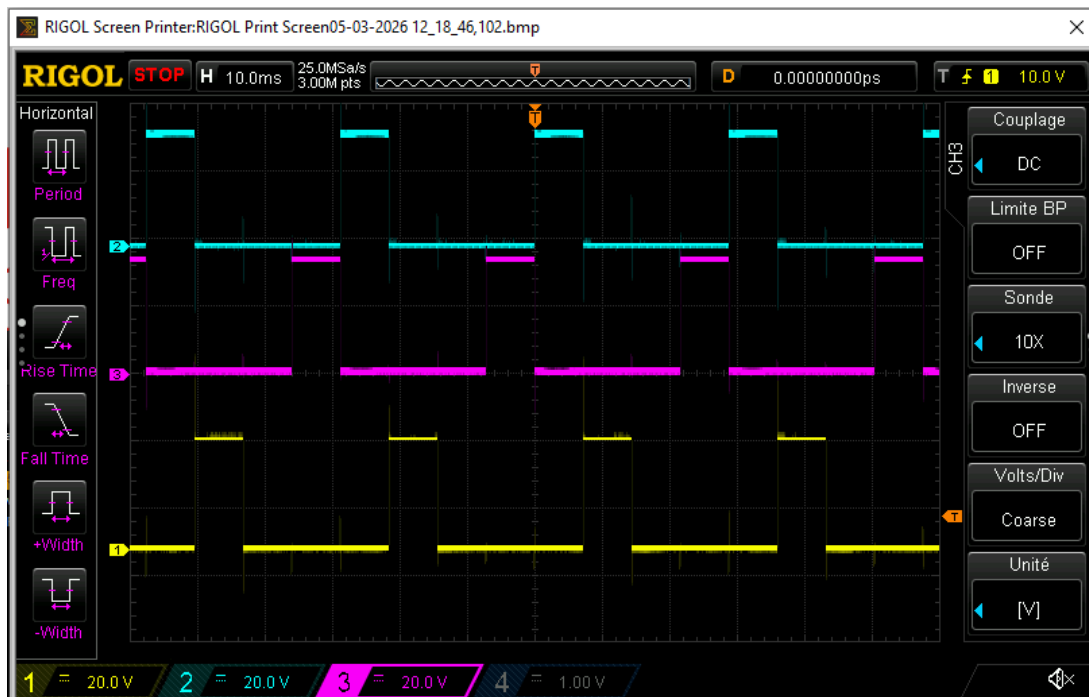


FIGURE 6 – Oscillograme des entrées Int 1..3 avec le code "Motor"

4 Capteur ultrason

Passons à l'étude du capteur à ultrasons HC-SR04. C'est un capteur très populaire, utilisé dans de nombreux projets Arduino pour sa simplicité et sa polyvalence. En consultant sa fiche technique (datasheet), nous avons obtenu les informations nécessaires à son implémentation dans notre projet.



FIGURE 7 – Capteur ultrasons HC-SR04

4.1 Principe

Le fonctionnement de ce capteur repose sur une sortie appelée TRIG et une entrée appelée ECHO. Le principe est d'envoyer un signal sur la sortie TRIG pour qu'elle produise une impulsion sonore pendant une période très brève (que l'on peut d'ailleurs entendre si l'on s'approche assez). Le son voyage et rebondit sur les surfaces ; il se réfléchit et revient vers notre entrée ECHO. De cette manière, nous obtenons toutes les variables nécessaires pour effectuer nos propres calculs de conversion afin d'avoir une distance lisible. Ces variables sont basées sur le temps : combien de temps le son a-t-il

mis pour aller jusqu'à l'obstacle et revenir.

4.2 circuit

Cette fois, c'est très simple : VCC et GND sont directement connectés à leurs broches respectives sur le GPIO. TRIG et ECHO sont connectés respectivement aux broches AA16 et AB16.

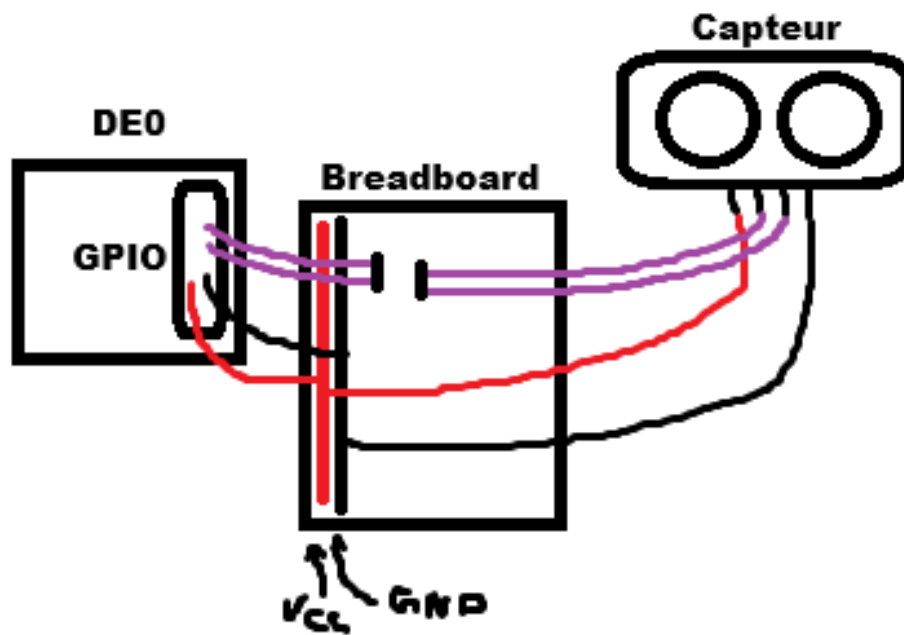


FIGURE 8 – Circuit Ultra

4.3 code VHDL

En commençant par les ports utilisés, la plupart sont logiques : deux entrées (l'horloge clock et echo qui reçoit le signal), et deux sorties pour Trig et une LED de vérification que nous avons assignée. Nous ajoutons également dist-cm sur 8 bits pour pouvoir afficher la valeur de la distance en cm après conversion.

Nous introduisons quelques signaux : trois compteurs et une variable de mémoire d'état end-echo, qui conserve l'état précédent

d'ECHO pour effectuer une manipulation que nous expliquerons plus tard.

Comme nous utilisons du 50 MHz comme entrée d'horloge dans notre circuit, ce qui correspond à une période de 20 ns, nous voulons que cntr-trig soit basé sur cela et s'incrémente avec l'horloge. Nous le réinitialisons chaque fois qu'il atteint 50 millions (soit 1 seconde). Nous utiliserons cela pour indiquer à notre TRIG le temps pendant lequel il enverra l'impulsion sonore ; nous avons conçu cela comme un rapport cyclique très bas.

Nous avons lu sur la fiche technique du HC-SR04 que l'impulsion doit durer environ 10 μ s. Pour obtenir cela, nous fixons TRIG à '1' pendant 500 cycles de notre compteur TRIG, car :

$$500 \text{ cycles} \times 20 \text{ ns}(50 \text{ MHz}) = 10\mu\text{s}$$

Pour le reste de cette seconde, TRIG reste à l'état bas.

Passons à la partie ECHO de notre code. Comme mentionné précédemment, end-echo conserve la valeur d'ECHO du cycle précédent. Avec ces deux variables, nous définissons quand démarrer ou arrêter le minuteur cntr-echo, qui déterminera à son tour la distance. Si ECHO est actif ('1'), cela signifie que le son voyage toujours, donc le compteur s'incrémente.

Si ECHO passe à '0' alors que l'état précédent était à '1' (c'est pourquoi nous avons end-echo, pour confirmer l'instant précis où arrêter le compteur), la division par 50 permet de convertir en μ s. Cette valeur est plus facile à manipuler pour fixer des conditions. La valeur est attribuée à count afin que nous puissions la comparer à des seuils à la fin. Nous avons ensuite donné au port dist-cm la valeur de count * 10 / 588 (nous n'utilisons pas 58,8 pour éviter les nombres réels) afin de convertir en cm. Nous avons trouvé cette for-

mule en ligne. Nous plaçons cette valeur dans `to-unsigned(valeur-cm, 8)` pour créer un code binaire de 8 bits représentant notre distance, que nous afficherons sur les afficheurs 7 segments. Nous terminons cette section en réinitialisant `cntr-echo`.

Enfin, nous avons défini une condition pour notre LED de confirmation en comparant la valeur de `count` avec des seuils choisis : un minimum d'environ un centimètre et un maximum de 42 cm pour que la LED s'éteigne, sinon elle reste allumée.

Nous avons utilisé un schéma bloc pour afficher les résultats sur les afficheurs 7 segments dans le projet "Ultraaffichage". La seule chose à noter est que nous avons utilisé "LPM-constant" via le MegaWizard pour créer une constante de 4 bits, afin d'afficher l'unité "cn" (cm) à côté de notre distance.


```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3  USE ieee.numeric_std.all;
4
5  ENTITY Ultra is
6  PORT ( clock: in std_logic;
7  LED : out std_logic;
8  TRIG : out std_logic;
9  ECHO : in std_logic;
10 dist_cm : out std_logic_vector(7 downto 0));
11 END Ultra;
12
13
14 architecture Behavioral of Ultra is
15 signal count      : integer := 0;
16 signal cntr_trig   : integer := 0;
17 signal cntr_echo   : integer := 0;
18 signal end_echo    : std_logic := '0';
19 begin
20
21 process(clock)
22
23 begin
24
25 if rising_edge(clock) then
26
27
28 if cntr_trig < 50000000 then
29     cntr_trig <= cntr_trig + 1;
30 else
31     cntr_trig <= 0;
32 end if;
33
34 if cntr_trig < 500 then
35     TRIG <= '1';
36 else
37     TRIG <= '0';
38 end if;
39
40 end_echo <= ECHO;
41 if ECHO = '1' then
42     cntr_echo <= cntr_echo + 1;
43     elsif ECHO = '0' and end_echo = '1' then
44         count <= cntr_echo / 50;
45         dist_cm <= std_logic_vector(to_unsigned((cntr_echo/50 * 10)
46 / 588, 8));
47         cntr_echo <= 0;
48 end if;
49
50 if count > 50 and count < 2500 then
51     LED <= '1';
52 else
53     LED <= '0';
54 end if;
55 end if;

```

```

56 end process;
57
58 end behavioral;

```

Listing 2 – Ultra VHDL code

Filter: Pins: all

		Node Name	Location
1		clk	PIN_G21
2		echo	PIN_AB16
3		led_conf	PIN_D13
4		sega[1]	PIN_E11
5		sega[2]	PIN_F11
6		sega[3]	PIN_H12
7		sega[4]	PIN_H13
8		sega[5]	PIN_G12
9		sega[6]	PIN_F12
10		sega[7]	PIN_F13
11		segb[1]	PIN_A13
12		segb[2]	PIN_B13
13		segb[3]	PIN_C13
14		segb[4]	PIN_A14
15		segb[5]	PIN_B14
16		segb[6]	PIN_E14
17		segb[7]	PIN_A15
18		segc[1]	PIN_D15
19		segc[2]	PIN_A16
20		segc[3]	PIN_B16
21		segc[4]	PIN_E15
22		segc[5]	PIN_A17
23		segc[6]	PIN_B17
24		segc[7]	PIN_F14
25		segd[1]	PIN_B18
26		segd[2]	PIN_F15
27		segd[3]	PIN_A19
28		segd[4]	PIN_B19
29		segd[5]	PIN_C19
30		segd[6]	PIN_D19
31		segd[7]	PIN_G15
32		trig	PIN_AA16
33		<<new node>>	

FIGURE 9 – Pin planner "Ultra"

4.4 Timing simulation

Encore une fois, pour produire une simulation, nous modifions certains de nos paramètres temporels, cette fois en augmentant le temps final (end time) à 1 ms, et en donnant à l'écho une période de 100 μ s, tout en modifiant également le timing interne du code, réduisant la condition du cntr-trig à 2000 et la condition d'activation du trig à 50. Ceci est pour la même raison que nous avons explorée avec la simulation temporelle du moteur. Nous obtenons ce qui suit :

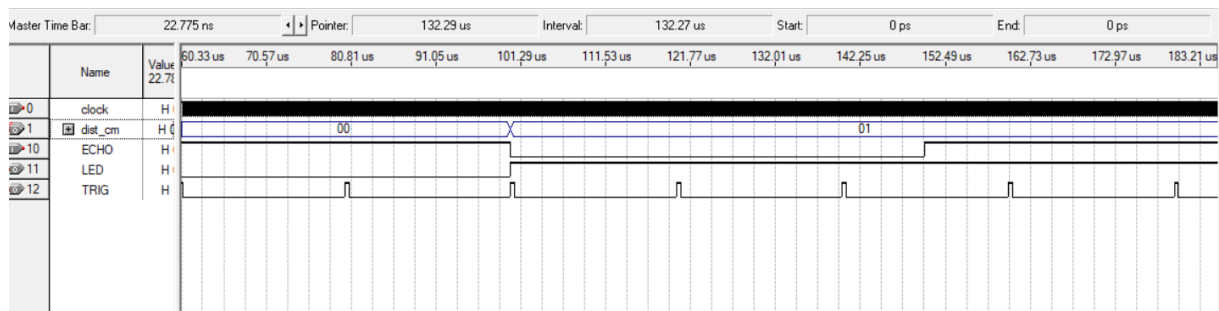


FIGURE 10 – Timing simulation "Ultra"

Comme prévu, les impulsions trig se répartissent sur toute la durée du test, pas de variation sur les afficheurs car il y a pas une variation de distance.

5 Communication entre le capteur et l'actionneur

5.1 Principe

L'approche que nous avons eue était très simple et directe : mettre le bloc principal du code du moteur dans la condition de distance, de sorte que si la distance est dans cet intervalle, le moteur tourne dans le sens des aiguilles d'une montre, sinon il tourne dans le sens inverse.

Nous avons également défini une condition spécifique pour le moteur, car nous ne voulons pas que la porte continue de s'ouvrir indéfiniment ; nous l'avons fait s'arrêter à un certain angle, ce qui serait l'état « ouvert ». Pour ce faire, nous avons bloqué le moteur à une valeur de pas ; pour obtenir la quantité de pas pour 90°, c'était simplement par essais et erreurs, jusqu'à ce que nous trouvions la valeur de 100, nous avons donc créé un signal nommé « max-step=100 ». Grâce à cela, nous avons également déterminé que notre moteur possède environ 400 pas pour un tour, bien que nous aurions probablement pu regarder la fiche technique pour cela. Pour utiliser cette nouvelle variable, nous ajoutons « cntr-steps » pour s'incrémenter à chaque fois qu'une séquence est complétée, afin de garder une trace de nos pas.

5.2 circuit

C'était assez direct, puisque nous avons déjà réalisé le circuit pour l'ultra et le moteur ; tout ce qu'il y a à noter est que nous avons défini une GND et un VCC communs sur la plaque d'essai, et que nous avons utilisé deux broches GPIO supplémentaires, comme on peut le voir sur la figure du Chip Planner.

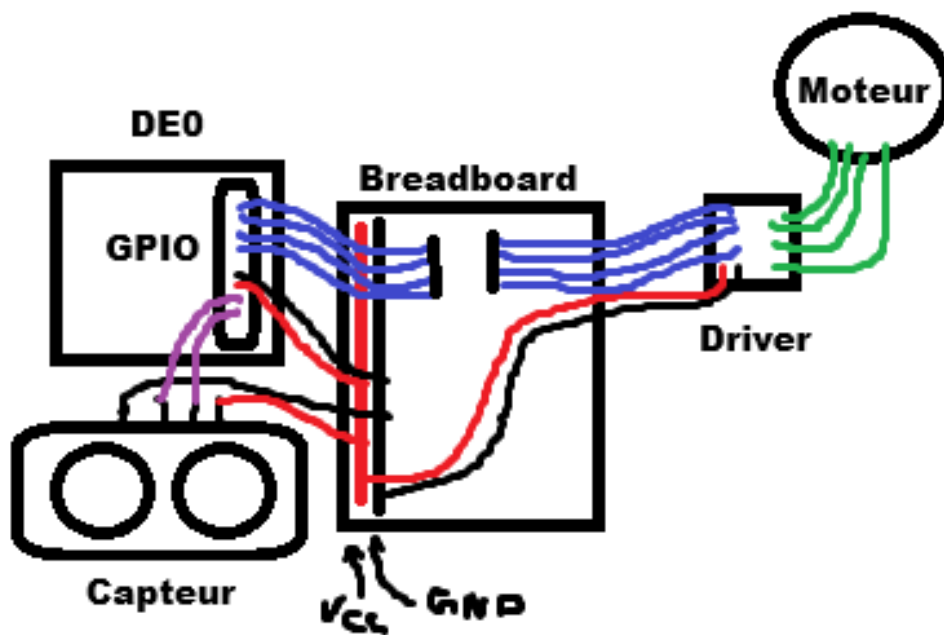


FIGURE 11 – Circuit door

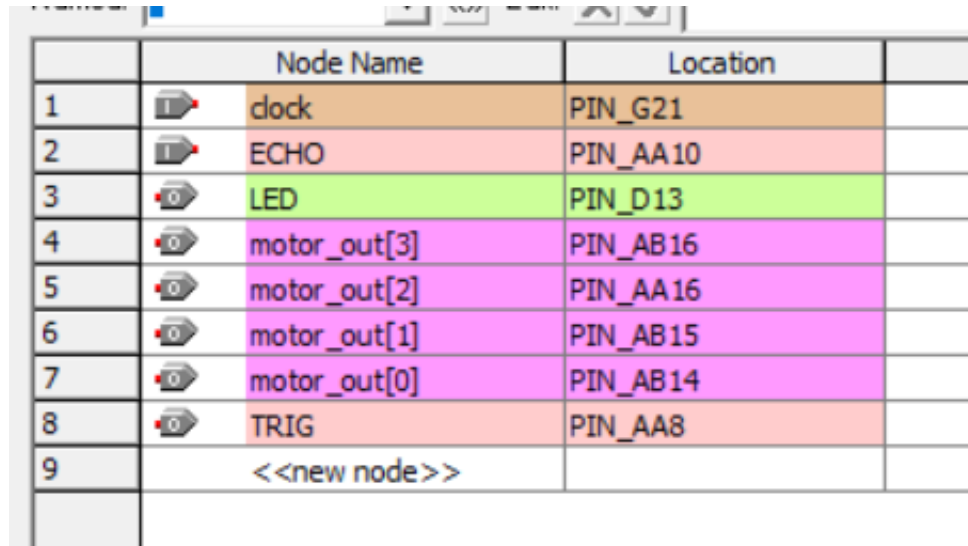
5.3 code VHDL

Maintenant, pour implémenter cela dans notre code, nous avons eu besoin de quelques autres modifications, principalement des instructions « if » et des changements de valeurs. Il est également important de noter que nous avons supprimé les parties inutiles de nos codes originaux « motor » et « ultra » qui n'étaient pas utiles ici, à savoir la variation de vitesse et l'affichage de la distance.

Nous avons changé les conditions de distance à 1 - 10 cm pour plus de précision, et nous avons réglé la vitesse (qui est la valeur du délai) à 720 kHz. Nous ajoutons une instruction « if » juste après la vérification de la distance, qui exécute un pas si $cntr_steps < max_step$ et incrémente $cntr_step$, et règle le moteur à « 0000 » sinon ; c'est ce qui arrête le moteur lorsqu'il atteint la limite de pas.

De même, pour que le moteur revienne en arrière, nous avons

défini une condition : si $\text{cntr_steps} > 0$, nous exécutons la séquence inverse et décrétons le compteur de pas; il s'arrête lorsqu'il atteint zéro.



	Node Name	Location
1	clock	PIN_G21
2	ECHO	PIN_AA10
3	LED	PIN_D13
4	motor_out[3]	PIN_AB16
5	motor_out[2]	PIN_AA16
6	motor_out[1]	PIN_AB15
7	motor_out[0]	PIN_AB14
8	TRIG	PIN_AA8
9	<<new node>>	

FIGURE 12 – Pin planner "door"

C'était simple de faire tous ces changements, maintenant notre moteur fonctionne comme on peut le voir dans la vidéo ci-dessous.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3
4
5  ENTITY door is
6  PORT ( clock: in std_logic;
7  motor_out: out std_logic_vector(3 downto 0);
8  LED : out std_logic;
9  TRIG : out std_logic;
10 ECHO : in std_logic);
11 END door;
12
13
14 architecture Behavioral of door is
15 signal max_step      : integer := 100;
16 signal speed         : integer := 720_000;
17 signal count         : integer := 0;
18 signal cntr_trig     : integer := 0;
19 signal cntr_echo     : integer := 0;
20 signal end_echo      : std_logic := '0';
21 signal cntr_steps : integer range 0 to max_step := 0;
22
23 begin
24
25

```

```
26 opening : process(clock)
27     variable cntr : integer := 0;
28
29
30 begin
31
32
33
34 if rising_edge(clock) then
35
36 if cntr_trig < 50000000 then
37     cntr_trig <= cntr_trig + 1;
38 else
39     cntr_trig <= 0;
40 end if;
41
42 if cntr_trig < 500 then
43     TRIG <= '1';
44 else
45     TRIG <= '0';
46 end if;
47
48 end_echo <= ECHO;
49 if ECHO = '1' then
50     cntr_echo <= cntr_echo + 1;
51 elsif ECHO = '0' and end_echo = '1' then
52     count <= cntr_echo / 50;
53     cntr_echo <= 0;
54 end if;
55
56
57 if count > 100 and count < 600 then
58     LED <= '1';
59
60     if cntr_steps < max_step then
61         cntr := cntr + 1;
62         if (cntr < speed*1/4) then
63             motor_out <= "1000";
64         elsif (cntr < speed*2/4) then
65             motor_out <= "0100";
66         elsif (cntr < speed*3/4) then
67             motor_out <= "0010";
68         elsif (cntr < speed*4/4) then
69             motor_out <= "0001";
70         else
71             cntr := 0;
72             cntr_steps <= cntr_steps + 1;
73         end if;
74     else
75         motor_out <= "0000";
76     end if;
77
78 else
79     LED <= '0';
80     if cntr_steps > 0 then
81         cntr := cntr + 1;
```

```
82  if (cntr < speed*1/4) then
83  motor_out <= "0001";
84  elsif (cntr < speed*2/4) then
85      motor_out <= "0010";
86  elsif (cntr < speed*3/4) then
87      motor_out <= "0100";
88  elsif (cntr < speed*4/4) then
89      motor_out <= "1000";
90  else
91      cntr := 0;
92      cntr_steps <= cntr_steps - 1;
93  end if;
94  else
95      motor_out <= "0000";
96  end if;
97
98 end if;
99
100 end if;
101 end process;
102
103 end behavioral;
```

Listing 3 – door VHDL code

5.4 Timing simulation

Pour obtenir un résultat cette fois-ci, nous réduisons les conditions du trig comme avant, nous réduisons à nouveau le délai du moteur, et nous augmentons le temps final (end time) de la simulation à 10 μ s. Nous avons également décidé de fixer la valeur de count à 300, ce qui est dans notre plage fonctionnelle, afin de pouvoir voir les résultats sur les moteurs.

On observe clairement la séquence du moteur, ainsi que la courte impulsion du trig comparée à la longue période de l'écho, ce qui nous permet de valider que tout est en ordre.

5.5 RTL viewer, Chip planner

Comme nous pouvons le voir dans les figures ci-dessous, notre système est principalement constitué de registres et de compteurs, avec quelques blocs d'addition simples pour l'incrémentation, quelques

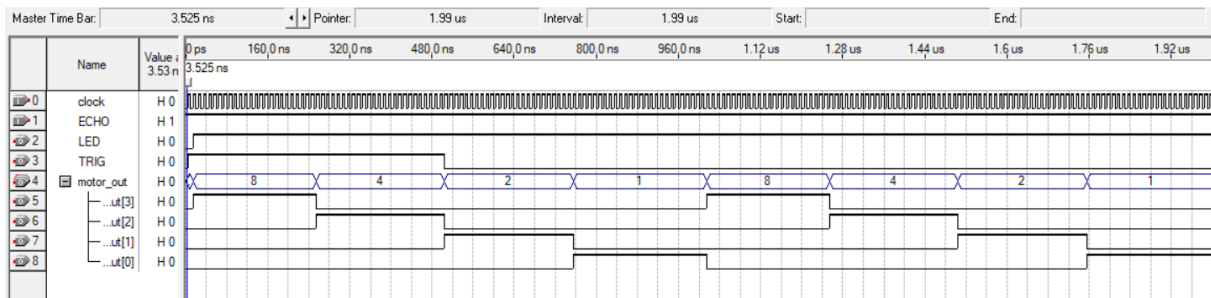


FIGURE 13 – Timing simulation "door"

blocs de comparaison pour les conditions des instructions « if », et un seul bloc logique « et » pour la double condition de distance. Ce qui est, dans l'ensemble, très simple et optimisé pour les fonctions de notre système ; nous avons le sentiment de n'avoir pas pris plus d'espace que nécessaire, ce qui est d'autant plus confirmé par le Chip Planner qui reste majoritairement vide, ainsi que par le rapport de compilation, utilisant 1 % des registres, 6 % des éléments logiques et 2 % des broches (8 broches). Nous considérons cela comme un succès.

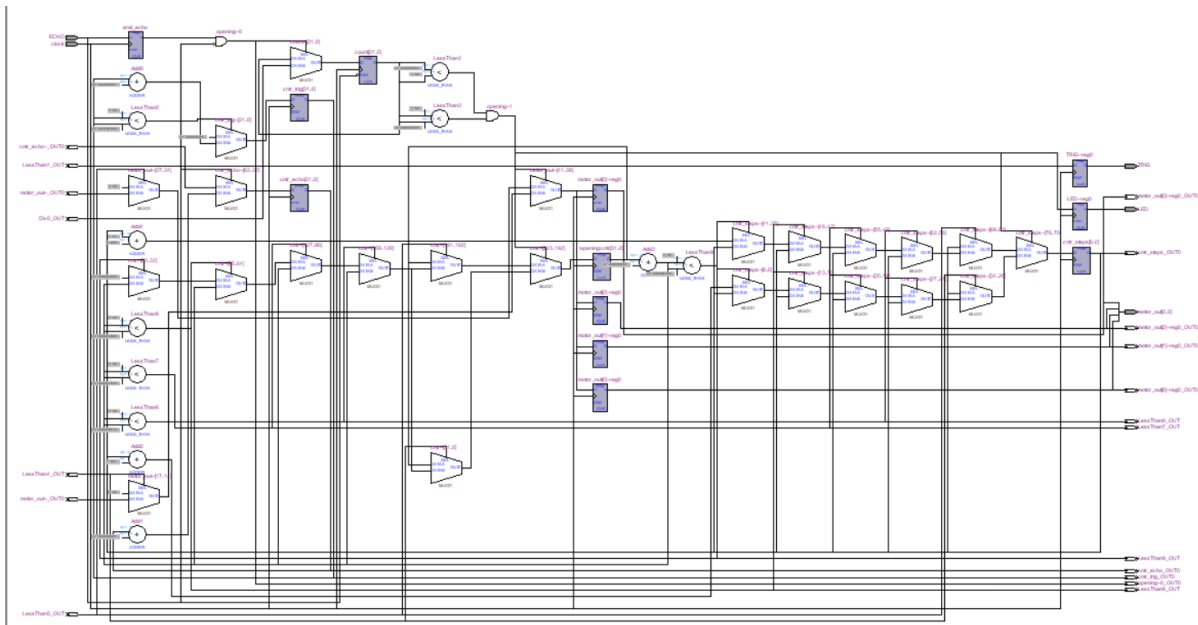


FIGURE 14 – Timing simulation "door"

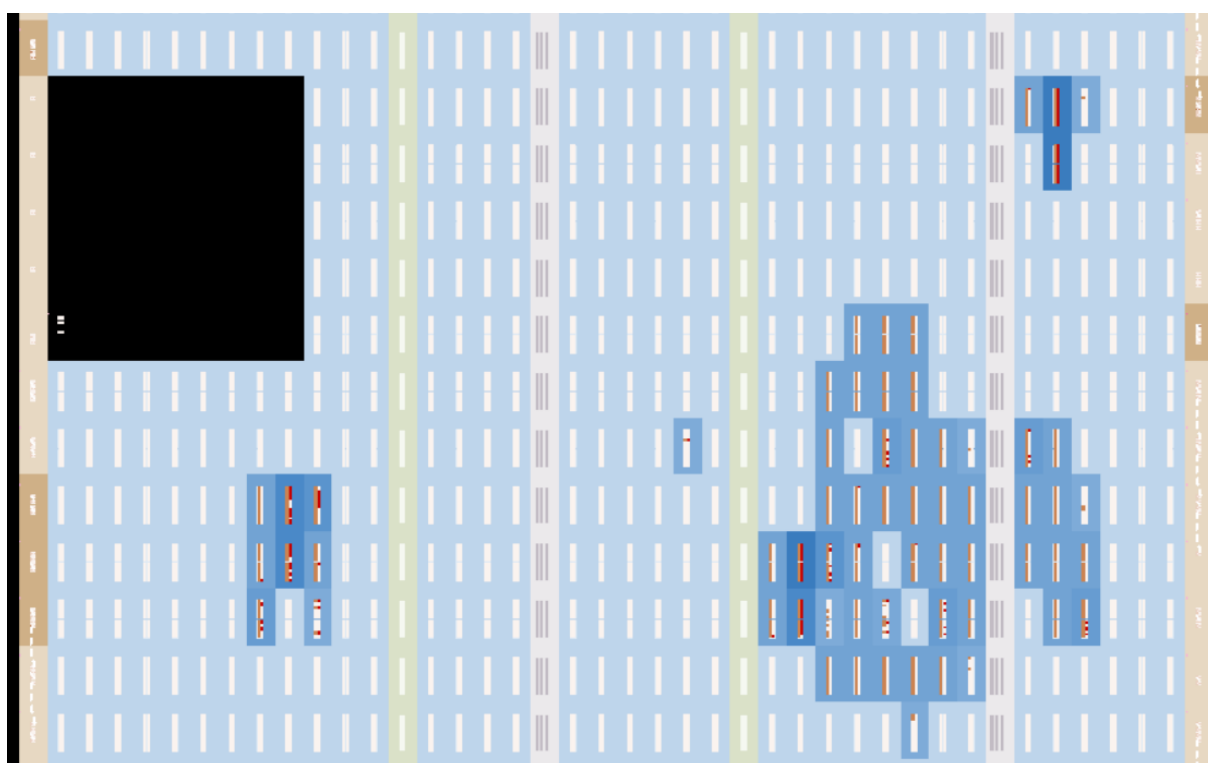


FIGURE 15 – Timing simulation "door"

6 Implémentation sur la carte DE10 Lite

Après nous être assurés que tout fonctionnait, il était temps de vérifier si nous étions capables de transférer notre travail sur une autre carte qui nous a été donnée, la **DE10-Lite**. Elle n'est pas de la même famille que la DE0, passant de Cyclone 3 à **MAX 10**, ce qui signifie que nous passons à un matériel plus moderne, impliquant également l'utilisation d'un logiciel plus récent ; nous avons donc dû télécharger **Quartus Prime Lite v20.1.1**. Nous avons d'ailleurs essayé de télécharger la version complète, sans réaliser qu'elle n'était pas gratuite et qu'elle ne produisait donc pas le fichier « .sof » nécessaire pour programmer notre carte.

L'interface était très similaire à la version de Quartus II que nous utilisions, donc la manipulation a été assez simple : nous avons ouvert un nouveau projet et défini le dispositif choisi sur « **10M50DAF484C7G** ». Nous avons collé notre code dans des projets du même nom. Nous avons dû consulter le Pin Planner pour la DE10-Lite sur sa fiche technique afin d'assigner nos ports. Une chose à laquelle nous ne nous attendions pas : le temps de compilation était très lent, le code VHDL n'étant peut-être pas optimisé pour cette version plus récente. Nous avons tout de même réussi à faire fonctionner notre carte et avons remarqué peu ou pas de différence dans le comportement de notre système.

Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate
clock	Input	PIN_P11	3	B3_N0	PIN_P11	2.5 V		12mA (default)	
ECHO	Input	PIN_AA15	4	B4_N0	PIN_AA15	2.5 V		12mA (default)	
LED	Output	PIN_A8	7	B7_N0	PIN_A8	2.5 V		12mA (default)	2 (default)
motor_out[3]	Output	PIN_W10	3	B3_N0	PIN_W10	2.5 V		12mA (default)	2 (default)
motor_out[2]	Output	PIN_W9	3	B3_N0	PIN_W9	2.5 V		12mA (default)	2 (default)
motor_out[1]	Output	PIN_W8	3	B3_N0	PIN_W8	2.5 V		12mA (default)	2 (default)
motor_out[0]	Output	PIN_W7	3	B3_N0	PIN_W7	2.5 V		12mA (default)	2 (default)
TRIG	Output	PIN_W13	4	B4_N0	PIN_W13	2.5 V		12mA (default)	2 (default)

FIGURE 16 – Ports sur "door" pour la carte DE10 lite

7 Conclusion

Ce test final de capacité qui nous a été confié représentait un défi amusant pour mettre en pratique nos connaissances sur les FPGA et le codage VHDL. Nous avons eu une approche méthodique, en séparant le moteur et le capteur ; après une étude individuelle, nous les avons combinés, analysé nos résultats, puis, comme cerise sur le gâteau, nous avons traduit notre travail sur une toute nouvelle carte avec un nouveau logiciel. Nous avons appris à naviguer dans un domaine inconnu, en découvrant les parties petit à petit pour obtenir un résultat final clair ; en parcourant les fiches techniques et les manuels, nous avons réussi à le faire fonctionner.

Table des figures

1	Chip planner scomp vs. scompplus	5
2	step motor 28BYJ-48	5
3	Circuit Moteur	7
4	pin planner "Motor"	9
5	Timing simulation "Motor"	11
6	Oscillograme des entrées Int 1..3 avec le code "Motor"	12
7	Capteur ultrasons HC-SR04	13
8	Circuit Ultra	14
9	Pin planner "Ultra"	18
10	Timing simulation "Ultra"	19
11	Circuit door	21
12	Pin planner "door"	22
13	Timing simulation "door"	25
14	Timing simulation "door"	25
15	Timing simulation "door"	26
16	Ports sur "door" pour la carte DE10 lite	27

Listings

1	Motor VHDL code	9
2	Ultra VHDL code	17
3	door VHDL code	22

8 Sources :

Datasheets :

- HC-SR04
- ULN2003 chip
- Step motor 28BYJ-48
- Quartus prime lite download
- Manuel DE10-Lite
- Manuel DE0